



DAY 4

Python Functions

Mastering Functions, Lambdas, Scope & Functional Programming



Today's Agenda

Part 1

- ◆ Introduction & Syntax
- ◆ Parameters & Arguments
- ◆ Return Values

Part 2

- ◆ Scope & Variables
- ◆ Nested & Recursive Fns
- ◆ Lambda Functions

Part 3

- ◆ map / filter / reduce
- ◆ Built-in vs User-defined
- ◆ Mini Projects



What is a Function?

A reusable block of code that performs a specific task



What is a Function?

Reusability

Write once, call many times. DRY principle.

Modularity

Break big problems into smaller, manageable pieces.

Readability

Named functions make code self-documenting.

Abstraction

Hide complexity behind a simple interface.

```
# Simple function example
def greet(name):
    """Return a greeting message."""
    return f"Hello, {name}! Welcome to Python."

# Calling the function
message = greet("Alice")
print(message)
# Output: Hello, Alice! Welcome to Python.
```



Functions are the building blocks of any Python program.



Function Syntax & Definition

def keyword

Declares a function

function name

Follows naming conventions

parameters

Input values (optional)

colon :

Ends the header

```
def function_name(param1, param2):  
    """Docstring: describes what this function does."""  
    # Function body (indented 4 spaces)  
    result = param1 + param2  
    return result          # optional return statement
```

— Syntax Rules

- # 1. Always start with def
- # 2. Name uses snake_case
- # 3. Parameters go inside ()
- # 4. Body must be indented
- # 5. return is optional (returns None by default)

Key Rules

- ✓ Use snake_case for names
- ✓ Docstring right after def
- ✓ Indentation is mandatory
- ✓ return exits immediately
- ✓ Can return any data type



Parameters & Arguments

Positional

```
def add(a, b):  
    return a + b  
add(3, 5) # a=3, b=5
```

Keyword

```
def greet(name, msg):  
    print(f"{msg} {name}")  
greet(msg="Hi", name="Bob")
```

Default

```
def power(base, exp=2):  
    return base ** exp  
power(3) # → 9  
power(3, 3) # → 27
```

*args

```
def total(*nums):  
    return sum(nums)  
total(1, 2, 3, 4) # → 10
```

**kwargs

```
def profile(**info):  
    for k, v in info.items():  
        print(f"{k}: {v}")  
profile(name="Alice", age=22)
```



Return Values & Multiple Returns

```
# Single return value
def square(n):
    return n * n

# Multiple return values (returns a tuple)
def min_max(lst):
    return min(lst), max(lst)

lo, hi = min_max([5, 2, 9, 1, 7])
print(lo, hi)    # 1 9

# Return dictionary
def stats(lst):
    return {
        "mean": sum(lst) / len(lst),
        "min": min(lst),
        "max": max(lst),
        "count": len(lst),
    }

result = stats([10, 20, 30, 40])
print(result["mean"])    # 25.0
```

return exits immediately

Code after return is never executed in that call.

No return → None

Python implicitly returns None if no return statement is used.

Tuple unpacking

Return multiple values; unpack cleanly: a, b = fn()

Return any type

int, str, list, dict, tuple, even another function!



Scope of Variables

L

Local

Variables defined inside the current function

E

Enclosing

Variables in outer function (nested functions)

G

Global

Variables defined at module / file level

B

Built-in

Python's built-in names: len, range, print ...

```
x = "global"           # Global scope

def outer():
    x = "enclosing"     # Enclosing scope

    def inner():
        x = "local"     # Local scope
        print(x)        # local

    inner()
    print(x)            # enclosing

outer()
print(x)               # global

# — global keyword —————
count = 0
def increment():
    global count
    count += 1          # modifies global

increment()
print(count)           # 1
```



Nested & Recursive Functions

— Nested Functions —

```
def outer(x):  
    def inner(y):  
        return x + y  
    return inner
```

defined inside outer
can access x (closure)
returns the function!

```
add5 = outer(5)  
print(add5(3))    # 8    (5+3)
```

— Recursive Functions —

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

base case (MUST have one!)
recursive call

```
print(factorial(5))    # 120  
# 5 × 4 × 3 × 2 × 1 = 120
```

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n-1) + fibonacci(n-2)
```

Closure

Inner functions remember variables from their enclosing scope.

Base Case

Every recursive function MUST have a base case to stop recursion.

Stack Depth

Python default recursion limit is 1000. Use `sys.setrecursionlimit()` if needed.

Memoization

Cache recursive results with `@functools.lru_cache` to boost speed.

λ Lambda / Anonymous Functions

lambda

x, y

:

x + y

keyword

parameters

separator

expression

```
# Regular vs Lambda
def add(a, b): return a + b
add_lambda = lambda a, b: a + b    # equivalent

# Common lambda uses
double = lambda x: x * 2
square = lambda x: x ** 2
is_even = lambda n: n % 2 == 0

print(double(5))    # 10
print(is_even(4))   # True

# Sorted with lambda key
students = [("Alice", 85), ("Bob", 72), ("Eve", 91)]
students.sort(key=lambda s: s[1])  # sort by grade
print(students)
# [('Bob', 72), ('Alice', 85), ('Eve', 91)]
```

```
# Conditional expression in lambda
```

Short one-liners

Ideal when a function is used only once.

As sort keys

lambda s: s[1] sorts by second element.

With map/filter

Perfect companion for functional tools.

Avoid complexity

Use def for anything needing >1 expression.



map · filter · reduce



map()

Applies a function to every item in an iterable.

```
nums = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x*2, nums))
# [2, 4, 6, 8, 10]

# With a named function
def celsius_to_f(c): return c*9/5 + 32
temps = list(map(celsius_to_f, [0, 20, 37, 100]))
```



filter()

Keeps only items where function returns True.

```
nums = range(1, 11)
evens = list(filter(lambda n: n%2==0, nums))
# [2, 4, 6, 8, 10]

names = ["Alice", "Bob", "Eve", "Anna"]
a_names = list(filter(lambda n: n[0]=="A", names))
# ['Alice', 'Anna']
```



reduce()

Cumulatively applies function to reduce to one value.

```
from functools import reduce
nums = [1, 2, 3, 4, 5]

product = reduce(lambda a,b: a*b, nums) # 120
total    = reduce(lambda a,b: a+b, nums) # 15

# Chain all three!
result = reduce(lambda a,b: a+b,
                filter(lambda n: n%2==0,
                    map(lambda n: n**2, range(1, 6))))
```



Docstrings, Type Hints & Best Practices

```
def calculate_bmi(weight: float, height: float) -> float:
    """
    Calculate Body Mass Index (BMI).

    Args:
        weight (float): Weight in kilograms.
        height (float): Height in metres.

    Returns:
        float: BMI rounded to 2 decimal places.

    Raises:
        ValueError: If height is zero.

    Example:
        >>> calculate_bmi(70, 1.75)
        22.86
    """
    if height == 0:
        raise ValueError("Height cannot be zero.")
    return round(weight / height ** 2, 2)
```

Type Hints

def fn(x: int) -> str: makes intentions clear and enables IDE support.

Docstrings

Always describe purpose, args, returns, and examples for public functions.

Pure Functions

Same input always gives same output. No side effects. Easy to test.

Naming

Use verbs for functions: get_total(), validate_email(), calculate_tax()

Single Responsibility

Each function should do one thing and do it well.

Pure vs Impure Functions & Error Handling

✓ Pure Function

```
def add(a, b):  
    return a + b    # always same result  
                  # no side effects
```

Impure Function

```
total = 0  
def add_to_total(n):  
    global total  
    total += n    # modifies external state!
```

```
# — Error Handling in Functions —————  
def safe_divide(a, b):  
    try:  
        result = a / b  
    except ZeroDivisionError:  
        return "Error: Cannot divide by zero"  
    except TypeError as e:  
        return f"Error: {e}"  
    else:  
        return result    # runs if no exception  
    finally:  
        print("Division attempted")    # always runs
```



Industry Mini Project

Utility Toolkit — Build Real-World Calculators



Mini Project — Utility Toolkit



EMI Calculator

Calculate monthly loan instalment using principal, rate, and tenure. Apply formula-based functions with float operations and formatted output.

Assignment 1



Tax Calculator

Compute income tax using slab-based logic. Use conditional branching inside functions and return breakdown of taxable amounts.

Assignment 2



Attendance Calculator

Calculate attendance percentage. Determine if requirements are met and how many classes must be attended to reach target.

Assignment 3



Day 4 Summary — Key Takeaways

def

Declares a function; body must be indented

lambda

Anonymous one-line function: `lambda x: x*2`

Parameters

Positional · Keyword · Default · `*args` · `**kwargs`

map/filter/reduce

Functional tools for transforming iterables

return

Sends value back; returns `None` if omitted

Pure Functions

No side effects; same input → same output

Scope (LEGB)

Local → Enclosing → Global → Built-in

Docstrings

Document purpose, args, returns, examples

Recursion

Function calls itself; must have a base case

Error Handling

Wrap risky code in `try/except` inside functions



Happy Coding!

Practice makes perfect — build, break, and learn. 🚀